TECHNICAL DESIGN & PRODUCT REQUIREMENTS DOCUMENT (TDD/PRD)

Project Name: AegisPay AI (Pluggable Real-Time Fraud & Anomaly Detection Microservice)
Document Version: 1.0.0
Author / Lead Architect: Sriniketh Avadhanula
Status: Approved Specification
Architecture Type: Pluggable API Middleware / Microservice

---

1. Executive Summary & Vision

AegisPay AI is an ultra-low latency, pluggable microservice engineered to evaluate financial transaction risk in real time before payment authorization. Positioned as an asynchronous middleware layer between payment gateways (e.g., Stripe, Razorpay), banking infrastructures, and e-commerce platforms, AegisPay AI ingests multi-dimensional transaction payloads, runs anomaly detection algorithms, and returns actionable risk verdicts in under 15 milliseconds.

By combining an in-memory caching layer (Redis), supervised and unsupervised machine learning models, and a zero-friction 1-tap push notification fallback, AegisPay AI minimizes chargebacks while virtually eliminating checkout drop-off caused by unnecessary OTP verifications.

---

2. Product Architecture & Scope

2.1 Functional Scope

- **Universal Middleware Integration:** Exposes standardized REST and WebSocket interfaces for external payment mediators.
- **Low-Latency Inference:** Guarantees sub-20ms end-to-end evaluation latency.
- **Smart Friction Triage:** Replaces intrusive SMS OTPs with 1-tap biometrically authenticated push prompts for medium-risk anomalies.
- **Interactive Developer & Operations Portal:** Provides auto-generated OpenAPI (Swagger) documentation and a lightweight, sleek cybertech monitoring dashboard.

2.2 Out-of-Scope (Phase 1)

- Direct settlement of fiat currency or ledger accounting (handled exclusively by payment gateways).
- Physical POS terminal hardware integration.

---

3. Risk Decision Engine & Business Logic

The core decision engine categorizes transactions into three operational tiers based on calculated risk probability (0.0000 to 1.0000):

Risk Score Tier	Risk Level	System Action	Primary Operational Outcome
Score < 0.60	Low Risk	`APPROVED`	Instant checkout success without friction
0.60 ≤ Score ≤ 0.85	Medium Risk	`CHALLENGE_PUSH`	1-tap biometric push prompt verification
Score > 0.85	High Risk	`DECLINED_FRAUD`	Immediate hard block & webhook security alerts

3.1 Tier Breakdown

- **Low Risk:** Probability 0.0000 to < 0.6000. System Action: `APPROVED`. User / Gateway Experience: Instant checkout success without interruption.
- **Medium Risk:** Probability 0.6000 to 0.8500. System Action: `CHALLENGE_PUSH`. User / Gateway Experience: Triggers a 1-tap in-app push notification (*"Did you make this purchase? [Yes / No]"*). On "Yes", transaction completes seamlessly.
- **High Risk:** Probability > 0.8500. System Action: `DECLINED_FRAUD`. User / Gateway Experience: Transaction blocked immediately. Webhook triggers multi-channel alerts to bank risk ops and account holder.

---

4. Data Contracts & API Specifications

4.1 Request Payload (POST `/v1/evaluate-transaction`)


```
{
 "transaction_id": "tx_9876543210_sec",
 "account_id": "acc_usr_10293",
 "merchant": {
  "id": "merch_apple_store",
  "category_code": 5732
 },
 "amount": 2500.00,
 "currency": "USD",
 "timestamp": "2026-10-13T22:15:00Z",
 "device": {
  "device_fingerprint": "df_a8f9c011e23",
  "ip_address": "192.168.1.100",
  "user_agent": "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0...)"
 },
 "location": {
  "latitude": 37.7749,
  "longitude": -122.4194
```

```
  }
}
```

4.2 Response Payload (200 OK, Target Latency: <15ms)

```
{
  "eval_id": "ev_01H9Z2X...",
  "transaction_id": "tx_9876543210_sec",
  "risk_score": 0.7420,
  "verdict": "CHALLENGE_PUSH",
  "latency_ms": 11.4,
  "reasons": [
    "UNUSUALLY_HIGH_AMOUNT_FOR_ACCOUNT",
    "NEW_DEVICE_FINGERPRINT"
  ],
  "challenge_details": {
    "push_notification_id": "push_8829103",
    "timeout_seconds": 30
  }
}
```

---

5. Technical Stack & Infrastructure Architecture

- Payment Gateway / Client connects via REST POST or WebSocket Stream to FastAPI Async Middleware.
- FastAPI Async Middleware (Rate Limiter & Validator) connects to Redis In-Memory Data and XGBoost / Isolation Forest Model Inference.
- Redis handles Velocity Counters and Key Rate Limiting.
- ML Models are pre-loaded in memory.

5.1 Component Definitions

- **Backend Engine:** Python with **FastAPI** utilizing async/await routines for concurrency.
- **In-Memory Cache: Redis** for storing rolling velocity keys (e.g., card swipe counts per minute) and IP rate limiting.
- **ML Inference Engine:** Pre-compiled **XGBoost** and **Isolation Forest** pipelines loaded directly into memory at server startup.
- **Frontend Portal & Dashboard:**
    - **API Portal:** FastAPI native Swagger UI (`/docs`) for key management and interactive endpoint testing.
    - **Live Risk Center:** Single-page HTML5 / CSS3 / Vanilla JS application connected via WebSockets, styled in a high-contrast dark/cyan cybertech theme for zero-flicker 60 FPS streaming.

---

6. Security, Rate Limiting & Performance SLAs

- **Latency SLA:** 99th percentile response time must remain under **20ms**.
- **Rate Limiting:** Redis-enforced leaky bucket algorithm restricting rogue client keys to maximum 100 requests/second per IP/Token.
- **Data Privacy:** Full PCI-DSS compliance design; no raw primary account numbers (PAN) stored in plain text—all account identifiers use hashed tokens.